

Exploratory Anvil Project

Vector Dot-Product Accelerator: SystemVerilog, SVA, and Anvil

Technical Report

Intikhab Khursheed | 28 August 2026

1. Project Objective and Design

The project investigates how much timing safety can be obtained from Anvil's type system compared with a handwritten SystemVerilog design plus a manually written SVA specification. The required workflow was: build a working accelerator in SystemVerilog; specify its important safety properties; reimplement the design in Anvil; and compare both implementations under matched transactions.

The chosen accelerator computes a dot product over configurable vectors using 8-bit unsigned elements. The handwritten implementation uses four arithmetic lanes operating in parallel, a shared multi-cycle accumulator, a lane-valid mask for partial groups, and ready/valid backpressure on the input and output. The Anvil implementation intentionally uses a sequential microarchitecture: one element pair is received, multiplied, accumulated, and acknowledged before the next element is accepted. This makes the comparison useful because the designs are functionally equivalent at the transaction boundary while having different internal timing.

2. SystemVerilog Reference (Part A)

The reference RTL is ``systemverilog/dot_product.sv``. It moves through IDLE, LOAD, REDUCE, and OUTPUT states. Commands carry ``cmd_length`` and ``cmd_tag``; element pairs are accepted during LOAD; the four lane products are generated concurrently; REDUCE visits each lane product over multiple cycles; and OUTPUT holds the result until the downstream consumer accepts it. The lane-valid array prevents unused lanes in a partial final group from contributing to the sum. The implementation therefore exercises the exact combination requested by the brief: concurrency, a shared resource, partial vectors, and backpressure.

The reference testbench (``systemverilog/testbench.sv``) provides the functional baseline. A separate broken SV variant, ``dot_product_broken.sv``, removes the accumulator reset at a command boundary. That defect is intentionally used later to demonstrate what the SVA specification catches.

3. SVA Specification (Part B)

The project defines six executable properties in ``sva/properties.sv``, with their rationale in ``sva/PROPERTY_RATIONALE.md``. The properties cover output stability, inactive-lane exclusion, accumulator reset, output-state correctness, command/result exclusion, and input-state correctness. Together they protect local state, interface stability, resource exclusion, and transaction-level protocol behavior.

Property	What it protects	Observed role
P1	Result and tag stay stable under output backpressure.	Protects interface stability.
P2	Inactive final lanes do not change the accumulator.	Protects partial-group correctness.
P3	Accumulator is cleared at command acceptance.	Catches the broken reset design.
P4	<code>`out_valid`</code> only occurs in OUTPUT.	Protects result-state ownership.
P5	A pending result blocks a new command.	Prevents transaction overlap.
P6	Input is accepted only in LOAD.	Protects lane storage from stray writes.

4. Anvil Reimplementation and Generated RTL

The Anvil source is ``anvil/dot_product.anvil``. It defines structured messages for commands, element pairs, and tagged results, then communicates through three channels. The process stores the accumulator, remaining element count, saved tag, and a small state value. The central calculation is: receive one pair, widen the operands, multiply them, add the product to the accumulator, decrement the remaining count, and acknowledge the element.

A notable implementation detail was arithmetic width. The source ultimately uses 20-bit ``p_a`` and ``p_b`` operands created through equal-width concatenations before ``p_a * p_b``. This was necessary because earlier attempts using the raw 8-bit operands or unsuitable casting forms did not preserve the desired product width in the generated RTL. The final implementation therefore matches the 20-bit accumulator contract used by the project. The current compiler represents a binary multiplication wire using the maximum operand width; in this design both operands are 20 bits, so the generated product wire is also 20 bits, which is sufficient for 8-bit-by-8-bit multiplication and the chosen accumulator contract.

```
let p_a = #{4'd0, 4'd0, 4'd0, pair.a[4+:4], pair.a[0+:4]} >>
let p_b = #{4'd0, 4'd0, 4'd0, pair.b[4+:4], pair.b[0+:4]} >>
let prod = p_a * p_b >>
```

The generated file ``anvil/generated/dot_product_anvil.sv`` contains the corresponding arithmetic and channel handshake logic. The multiplication appears as a continuous assignment, while the event machinery controls when its result may update state or be presented at the interface.

```
assign thread_0_wire$38 = thread_0_wire$29 * thread_0_wire$37;
assign thread_0_wire$40 = thread_0_wire$39 + thread_0_wire$38;
```

5. Matched-Stimulus Differential Testing

The differential testbench is ``sva/testbench_equivalence.sv``. It uses one logical transaction source and two protocol adapters: a ready/valid adapter for the handwritten SV and a channel-handshake adapter for the Anvil-generated interface. Because the two architectures have different timing, the

comparison is transaction-level rather than cycle-for-cycle. For each transaction, the testbench checks the expected result independently on both implementations and then checks that the two observed results and tags agree.

The strengthened suite was executed in Verilator 5.044 through EDA Playground. The complete run passed seven tags/cases, including length 1, length 2 with zero operands, a length-3 partial group, length 4, multiple commands, and maximum operands with output backpressure. The maximum-operand case produced 260100 on both implementations, matching the expected value. The suite also records command and element handshake counts so the same requested transaction was accepted on both sides.

Case	Length / condition	Expected	Result
1	1	63	SV=63, Anvil=63 — PASS
2	2 + zero operands	0	SV=0, Anvil=0 — PASS
3	3 / partial group	32	SV=32, Anvil=32 — PASS
4	4	70	SV=70, Anvil=70 — PASS
5A	multiple commands	30	SV=30, Anvil=30 — PASS
5B	multiple commands	50	SV=50, Anvil=50 — PASS
6	max operands + backpressure	260100	SV=260100, Anvil=260100 — PASS

The comparison intentionally does not require internal accumulator or lane-state equality across the two architectures, because those states have different representations and update timing. Instead, lane behavior and accumulator safety are checked through the implementation-specific P2/P3-style invariants while cross-implementation equivalence is established at the observable transaction boundary. This is the technically meaningful boundary for two intentionally different microarchitectures.

6. What Anvil Accepts and What It Does Not Guarantee

The most informative result is that timing safety and functional correctness are different properties. Anvil's compiler checks the timing/lifetime discipline of a well-typed program, but it does not prove that the arithmetic computation implements the intended mathematical specification. The project demonstrates both sides with two negative experiments.

6.1 Anvil rejects a timing/lifetime violation

The file ``anvil/dot_product_rejected.anvil`` is a deliberately small request/response design that sends a value whose lifetime does not cover the required response send. Before running the compiler, the predicted failure was a borrow/lifetime rejection. The actual command was:

```
dune exec anvil -- -just-check D:\anvil-vector-dot-product\anvil\dot_product_rejected.anvil
```

The compiler rejected the program with:

```
Compilation failed!
Borrow checking failed:
Value does not live long enough in send!
```

No unrelated warnings were present in the final run. This provides direct evidence for the first required observation in the brief: a plausible design is rejected before SystemVerilog generation because its value cannot safely satisfy the timing/lifetime obligation of the send.

6.2 Anvil accepts a timing-safe but functionally wrong design

The file ``anvil/dot_product_drop_product.anvil`` preserves the normal channel structure and sequencing but changes one datapath operation from adding the product to adding only the first operand:

```
// Correct: set accum := *accum + prod >>
set accum := *accum + p_a >>
```

Anvil accepted this source with ``-just-check`` and therefore allowed it to be generated. The generated RTL contains the corresponding wrong arithmetic. When run through the matched differential environment, the wrong implementation produced 9 for the length-1 vector while the expected dot product was 63. This shows that the failure is semantic, not a timing failure: the protocol completes and the result is returned, but the arithmetic contract is violated.

The distinction is central to the project question. Anvil provides a strong compile-time timing guarantee, but functional intent still requires explicit specification and testing. In this experiment the functional/differential test suite identifies exactly the class of error that the timing type system is not designed to prevent.

7. Compiler Trace: From Anvil Source to SystemVerilog

The compiler trace uses the actual Anvil compiler revision ``d138cabedbf3b65c08249ce6a55cb90dad959da``. The traced construct is the multiplication ``let prod = p_a * p_b >>``. The path is:

```
dot_product.anvil
-> lib/compileDriver.ml
-> lib/graphBuilder.ml :: construct_graphIR
-> lib/wireCollection.ml :: add_binary / Wire.new_binary
-> lib/codegen.ml :: codegen_wire_assignment
-> generated dot_product_anvil.sv
```

In ``compileDriver.ml``, the compiler parses the source, builds event-graph collections, and invokes code generation unless ``-just-check`` is used. In ``graphBuilder.ml``, the ``Binop`` case recursively builds both operands and creates a binary wire. ``wireCollection.ml`` stores the operation as a ``Binary`` wire source and records its width. Finally, ``codegen.ml`` pattern-matches that wire and formats the binary operator as SystemVerilog. The generated assignment is therefore a direct manifestation of the compiler's intermediate wire representation.

8. Findings and Comparison with the Assignment Goal

The project completed the intended $A \rightarrow B \rightarrow C$ workflow and produced a controlled comparison between a conventional RTL implementation and an Anvil implementation. The SystemVerilog version demonstrates explicit microarchitectural timing and manual safety specification. The Anvil version expresses the same transaction-level function with channel-based communication and relies on the compiler to reject timing/lifetime violations.

Assignment requirement	Evidence in repository	Status
A. Working SystemVerilog design	<code>`systemverilog/dot_product.sv`</code> + functional testbench; concurrency and backpressure exercised.	Complete
B. SVA specification + rationale	<code>`sva/properties.sv`</code> + <code>`PROPERTY_RATIONALE.md`</code> ; six properties with rationale.	Complete
B. Broken design caught by a property	<code>`systemverilog/dot_product_broken.sv`</code> ; P3 detects missing reset.	Complete
C. Anvil port + generated RTL	<code>`anvil/dot_product.anvil`</code> + <code>`anvil/generated/dot_product_anvil.sv`</code> .	Complete
C. Matched stimulus / differential test	<code>`sva/testbench_equivalence.sv`</code> ; seven deterministic cases passed.	Complete
Observation 1: rejected by Anvil	<code>`observations/rejected_by_anvil.md`</code> ; borrow/lifetime rejection.	Complete
Observation 2: accepted but wrong	<code>`observations/accepted_but_wrong.md`</code> ; timing-safe datapath mutation caught functionally.	Complete
Observation 3: compiler trace	<code>`observations/compiler_trace.md`</code> ; source-to-codegen trace.	Complete
README + repository structure	<code>`README.md`</code> describes the flow and major files.	Complete

9. Conclusion

The main result is not simply that both implementations compute the same dot products. The more useful result is the separation of responsibilities. SystemVerilog plus SVA provides explicit functional and protocol specifications, while Anvil’s type/lifetime checking moves an important class of timing-safety errors into compilation. The rejected example shows this guarantee in action: a value-lifetime violation is detected before RTL generation. The accepted-but-wrong example shows the boundary of that guarantee: a program can remain timing-safe while implementing the wrong arithmetic.

The differential test then provides an independent bridge between the two implementations. It uses matched transaction data, verifies requested handshakes, checks each implementation against the mathematical expectation, and finally checks that the observable results and tags agree. Because the microarchitectures are different, the comparison is intentionally made at transaction boundaries rather than by demanding identical cycles or identical internal state. This makes the experiment a practical demonstration of how language-level timing guarantees and conventional functional verification complement each other.

10. What I Would Improve in Anvil

- Provide clearer arithmetic-width diagnostics and easier explicit widening/extension syntax for common RTL operations. In this project, multiplication required several iterations before arriving at a clean equal-width construction.
- Improve diagnostics for mixed concerns so that timing/lifetime errors are visually separated from ordinary type-width issues and the user can identify the primary cause immediately.
- Expose a simpler, human-readable mapping from source expressions to event-graph/code-generation nodes. The compiler trace is possible today, but it requires reading several OCaml modules.
- Provide a first-class verification story for functional intent. Anvil can guarantee timing safety, but the project demonstrates that arithmetic correctness still needs assertions, reference models, or differential testing.

11. Reproducibility and Artifacts

Primary artifacts are organized as: ``systemverilog/`` for the reference RTL and functional test, ``sva/`` for properties and verification benches, ``anvil/`` for source and generated RTL, ``tests/`` for the standalone generated-RTL test, and ``observations/`` for the three required investigations. The compiler revision used for the trace is recorded in ``observations/compiler_trace.md``, and the project README documents the main commands and file relationships.

The final evidence set therefore consists of source code, executable properties, generated RTL, negative examples, compiler diagnostics, and matched differential simulation results. This directly addresses the investigation question: Anvil removes a class of timing hazards from the manual verification burden, while functional correctness remains an independent obligation.

References

1. Exploratory Anvil Project brief, “A Vector or Dot-Product Accelerator in Anvil”, supplied by Dr. Umang Mathur.
2. Anvil compiler source, revision ``d138cabedbf3b65c08249ce6a55cb90dad959da``, especially ``compileDriver.ml``, ``graphBuilder.ml``, ``wireCollection.ml``, and ``codegen.ml``.
3. Project repository: <https://github.com/IntikhabKhursheed/anvil-vector-dot-product>